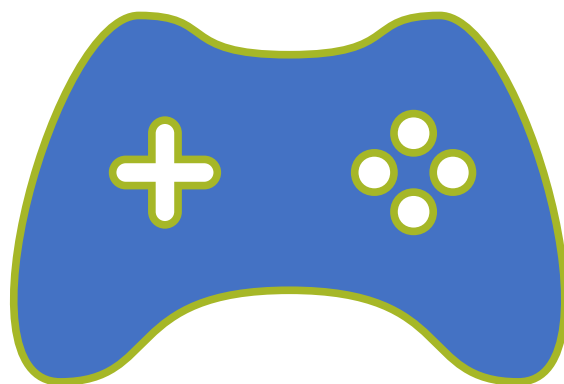




משחקים – חלק 2

פרק 5

מה נלמד היום ?

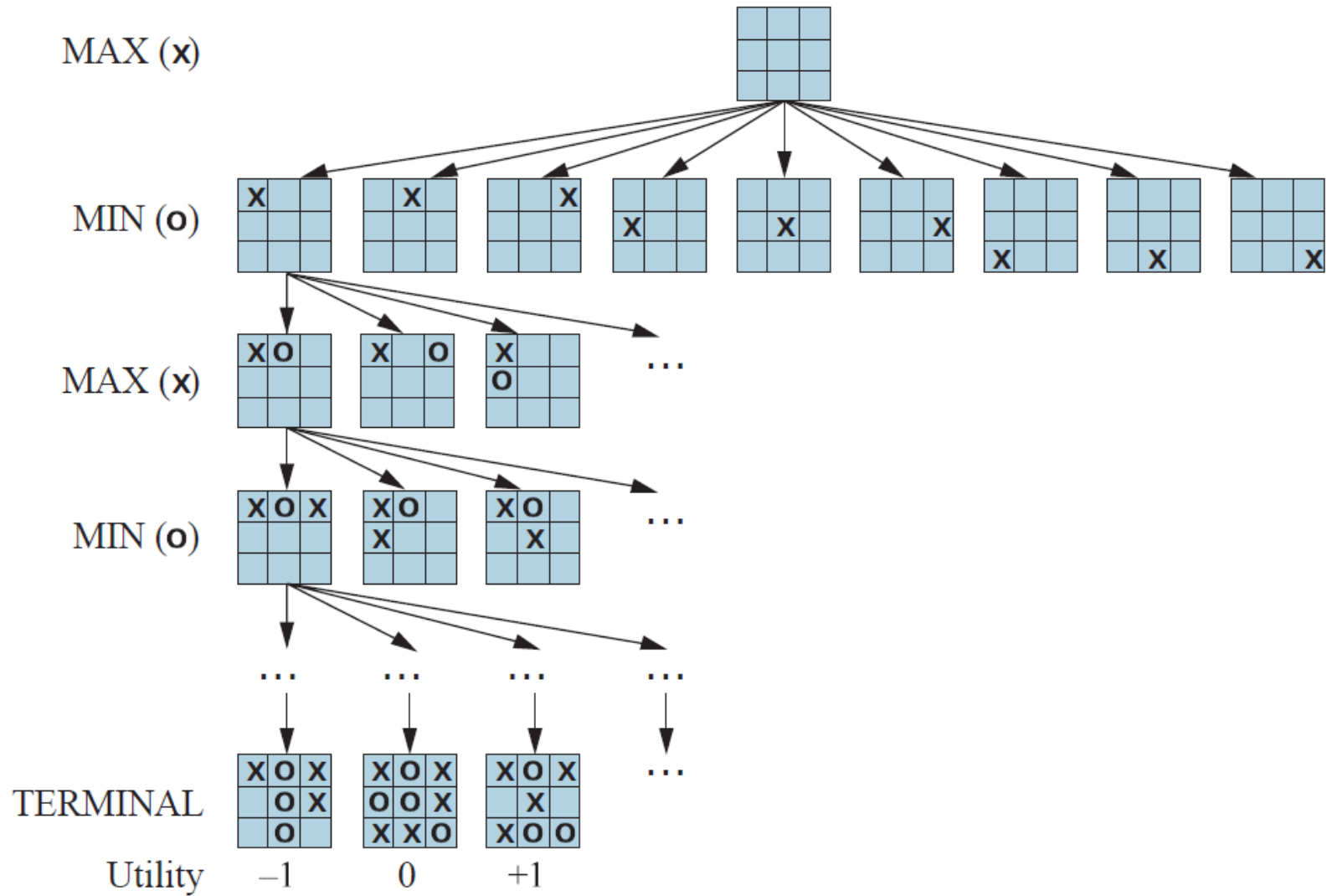
α - β pruning ★

Imperfect, real-time decisions ★

המצב כיום ★



עץ המשחק של איקס-עיגול מנקודת מבטו של X - שחקן MAX



Minimax

- הרעיון: לעבור לצעד שבו היריב יפגע בך הכי פחות,
צעד שערך ה minimax value שלו הגבוה ביותר.
• מניחים שהיריב משחק באופן אופטימאלי גם הוא.

הבעיה ב Minimax

מה הזמן והמקום הנדרשים עבור הרצת האלגוריתם?

For chess, $b \approx 35$, $m \approx 100$ for "reasonable" games

כיצד נשפר את האלגוריתם?

שיפור ביצועי Minimax

דרך א - לגזור תתי עצים שאינם משפיעים על החישוב.

α - β Pruning algorithm

דרך ב - לחסוך פיתוח של מצבים חוזרים

דרך ג – צימצום מושכל של עץ המשחק.



ALPHA- BETA PRUNING

α - β Pruning

הבעיה באלגוריתם Minmax היא שהוא דורש זמן אקספוננציאלי, לכן לא ניתן לרוב לפרוש את כל עץ המשחק ולקבל החלטה אופטימלית לפיו.

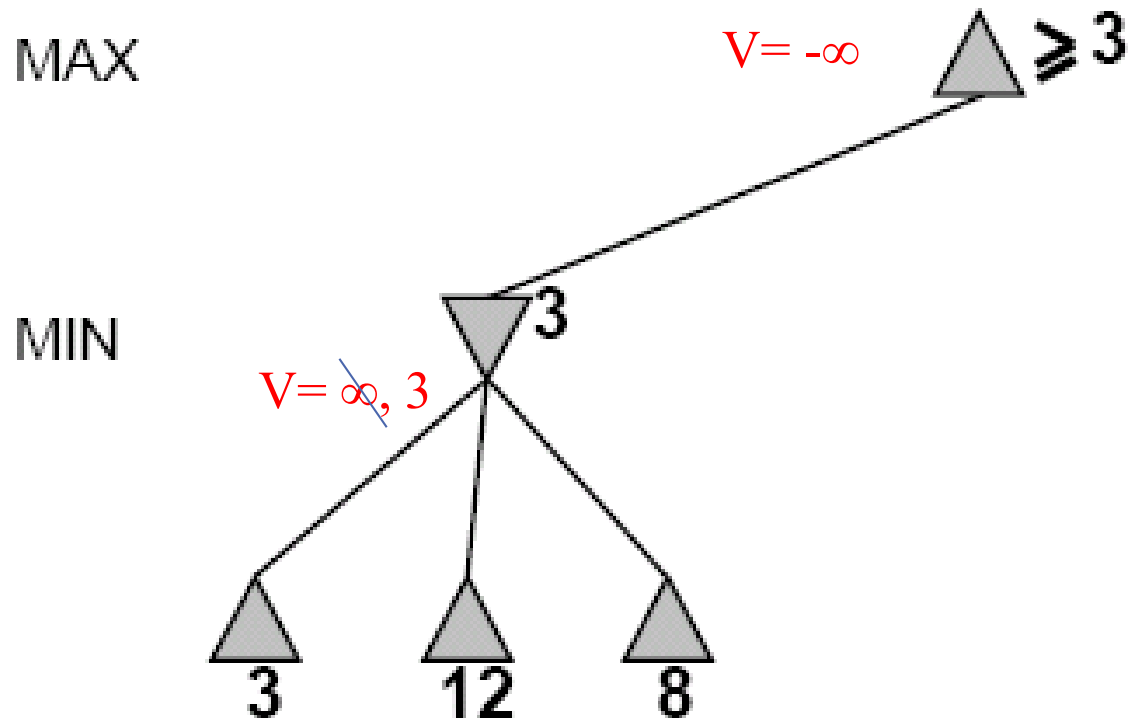
הצעה לשיפור: לגזום מהעץ, תוך כדי הפעלת Minmax, ענפים שבוודאי לא ישפיעו על ההחלטה הסופית.

כך ניתן לצמצם החזקה של b לכדי חצי משהייתה

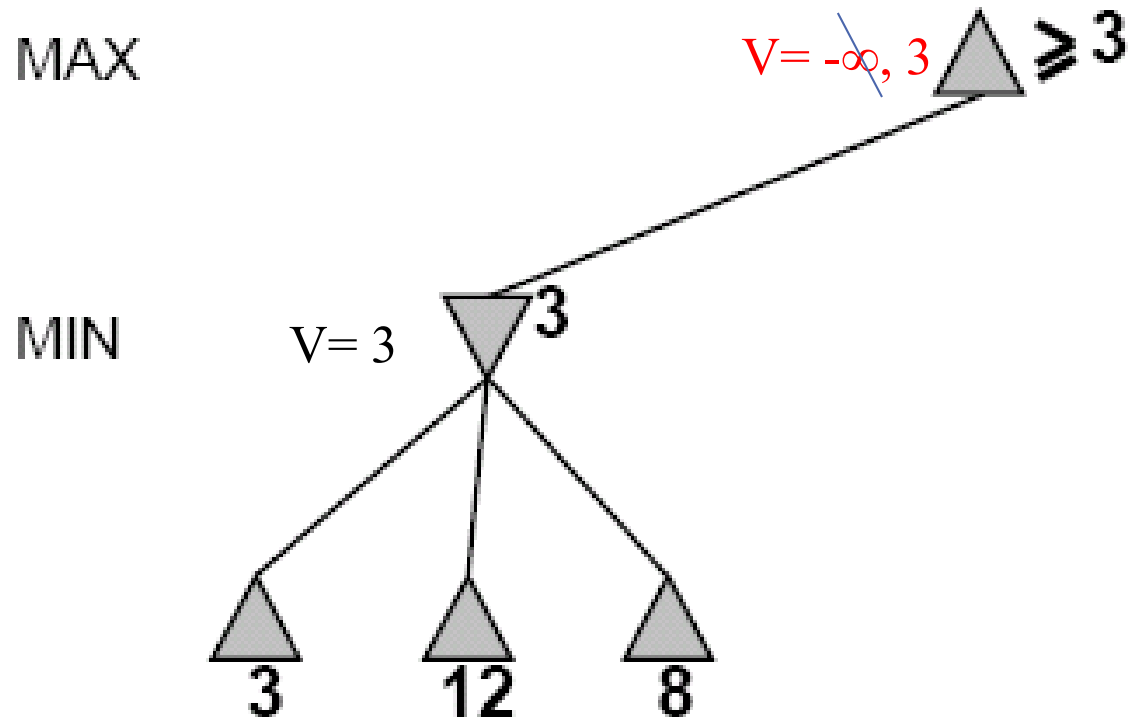
• [סרטון הסבר החל מדקה 5:20](https://www.youtube.com/watch?v=l-hh51ncgDI)

<https://www.youtube.com/watch?v=l-hh51ncgDI>

α - β Pruning Example



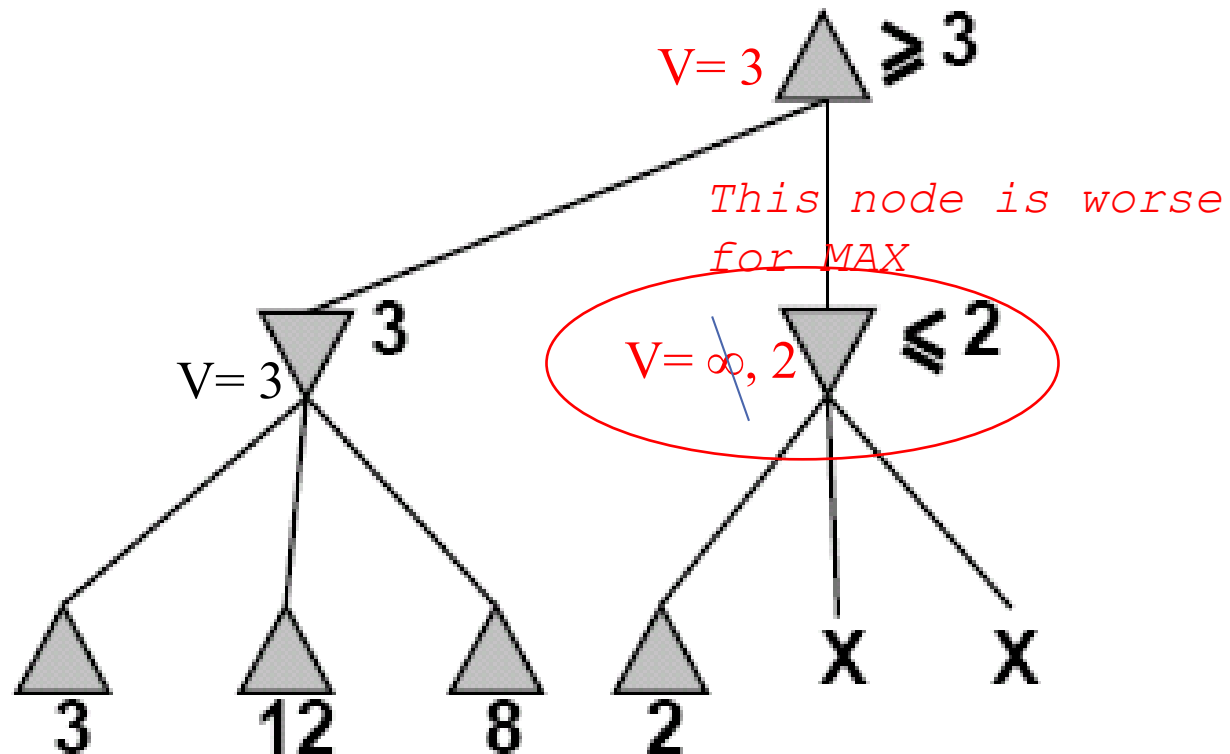
α - β Pruning Example



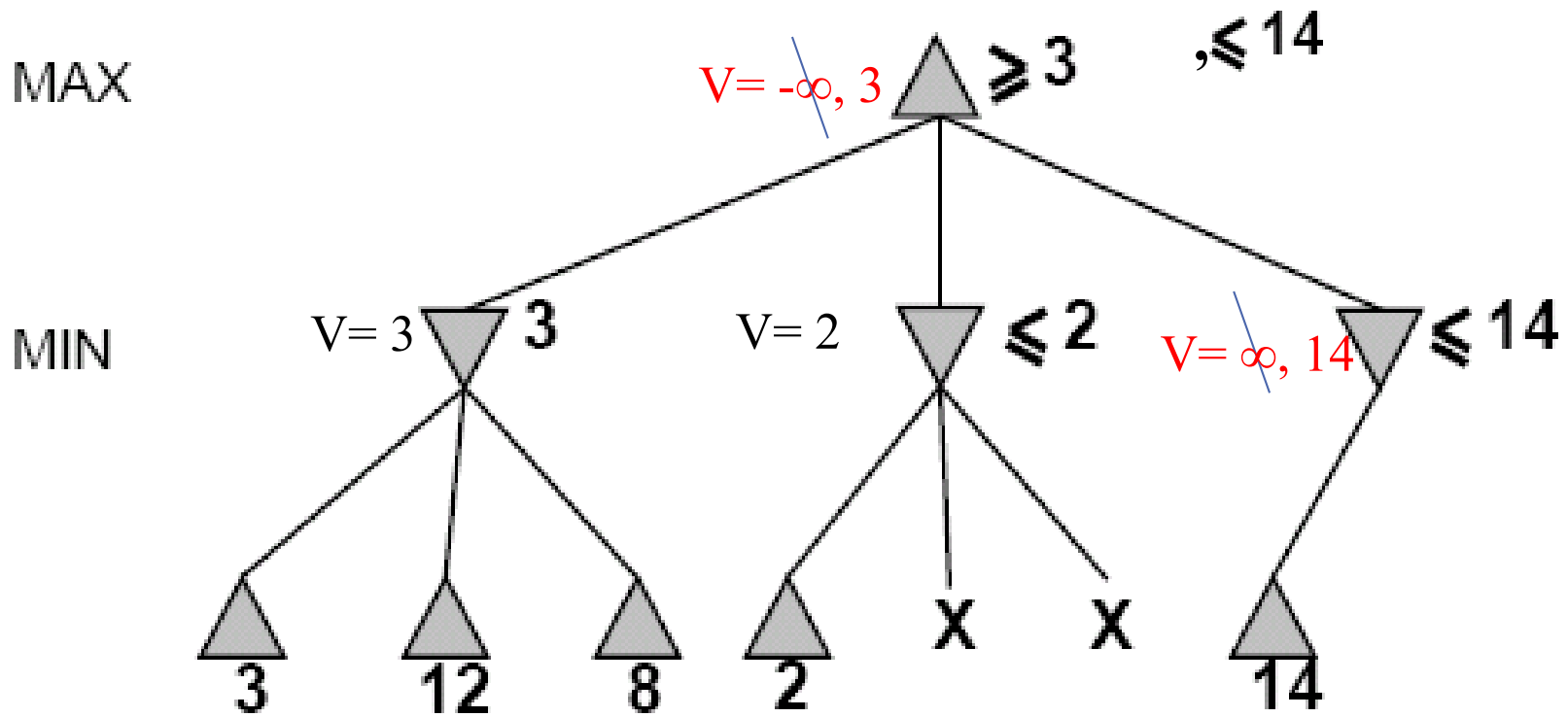
α - β Pruning Example

MAX

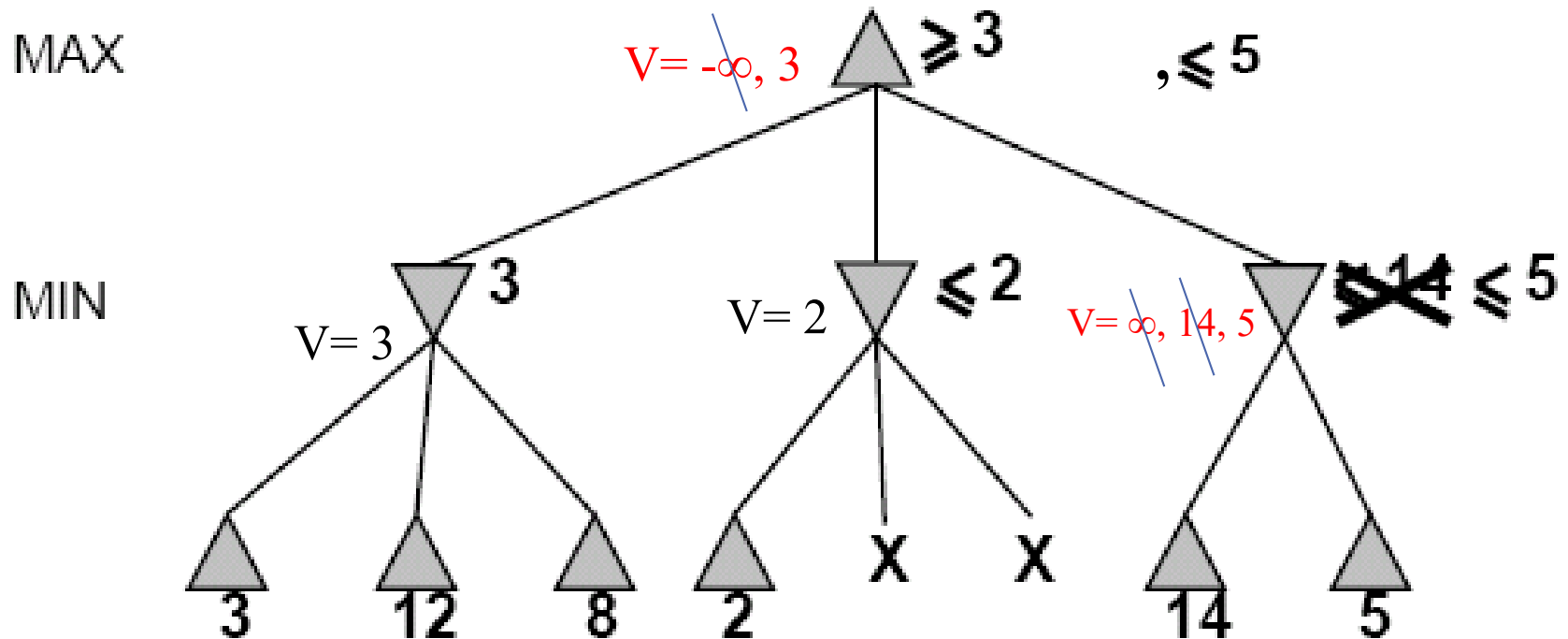
MIN



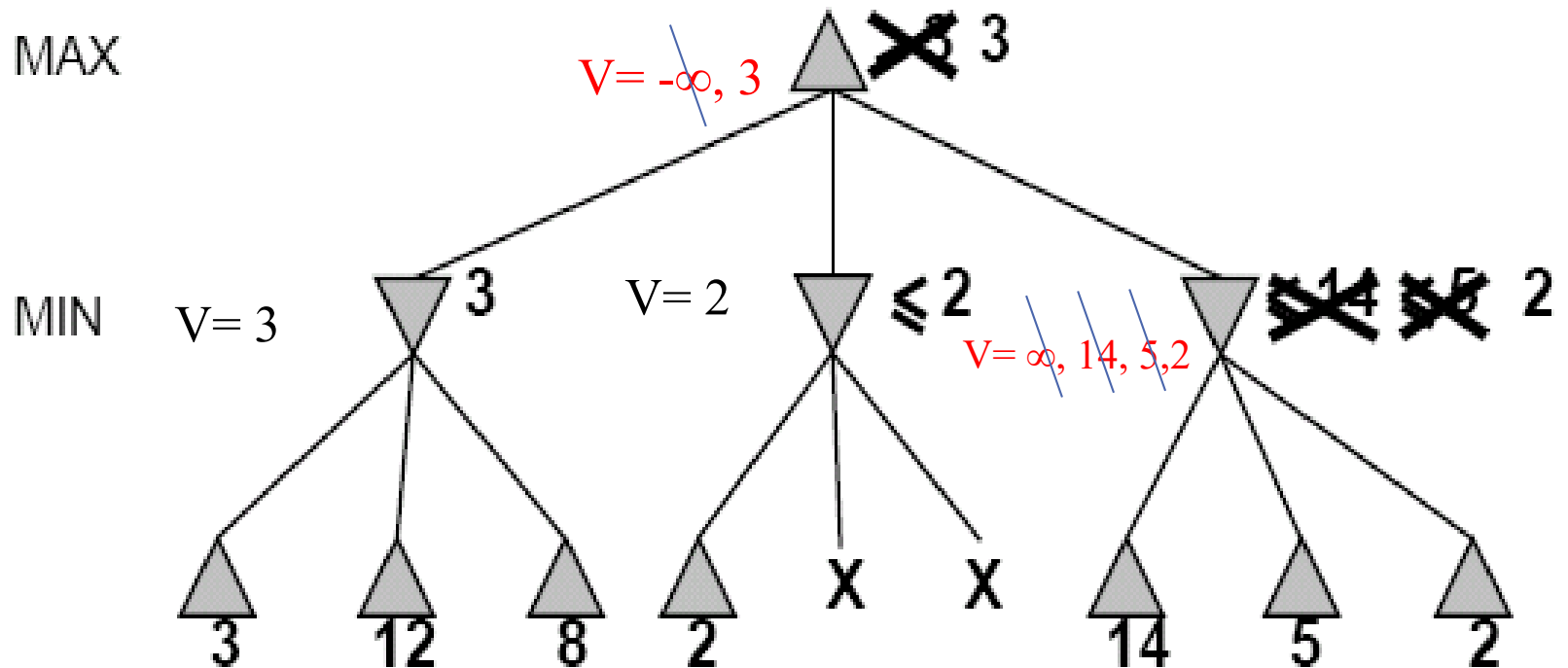
α - β Pruning Example



α - β Pruning Example



α - β Pruning Example



Why is it called alpha-beta?

α הינו הערך הכי טוב- הכי

גבוה ש Max ראה עד עכשיו בין

כל צאצאיו שנבדקו.

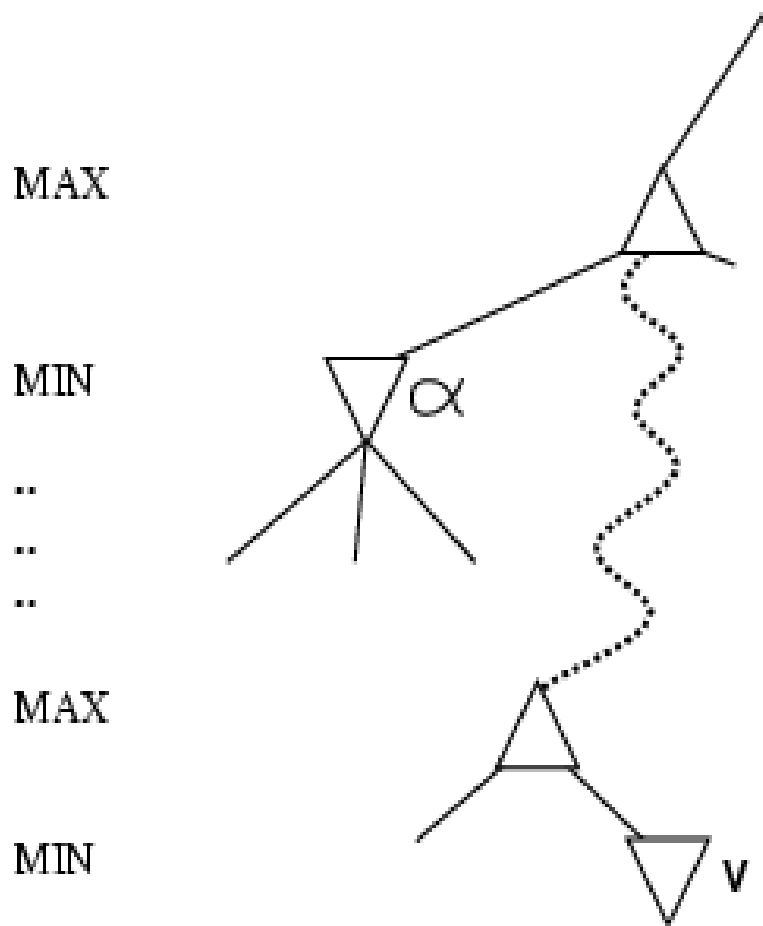
• אם כעת מגיעים לקוד' Min ויש

שם ב v ערך קטן מ α ניתן

להתעלם ממנו כלומר לגזום

את הענף של v .

• β בדומה עבור Min



The α - β algorithm

- ★ האלגוריתם עובד כמו אלגוריתם Minmax רק ששומר:
 - לכל קודקוד Max, במהלך החישוב ערך α שהינו הערך הגבוה ביותר שניתן למצוא עד כאן ע"י Max.
 - לכל קודקוד Min, במהלך החישוב ערך β שהינו הערך הנמוך ביותר שניתן למצוא עד כאן ע"י Min.

★ שחקן Min גוזם תתי עצים אם ערך ה val שלו קטן יותר מערך ה α של אביו Max.

★ שחקן MAX גוזם תתי עצים שלו אם ערך ה val שלו גדול יותר מערך ה β של אביו Min.


```

AlphaBetaSearch(state)                                //  $\alpha$  is best so far for Max
{
    v = MaxValue(state,  $-\infty$ ,  $\infty$ )                //  $\beta$  is best so far for Min
    return the action in Successors(state) with value v
}

```

```

MaxValue(state,  $\alpha$ ,  $\beta$ )
{
    if TerminalTest(state)
        return Utility(state)
    val =  $-\infty$ 
    For all s in Successors(state)
        {val = max {val, MinValue(s,  $\alpha$ ,  $\beta$ )}}
         $\alpha = \max \{\alpha, \text{val}\}$ 
        if  $\alpha \geq \beta$  return val
    }
    Return val
}

```

```

MinValue(state,  $\alpha$ ,  $\beta$ )
{
    if TerminalTest(state)
        return Utility(state)
    val =  $\infty$ 
    For all s in Successors(state)
        { val = Min {val, MaxValue(s,  $\alpha$ ,  $\beta$ )}}
         $\beta = \min \{\beta, \text{val}\}$ 
        if  $\beta \leq \alpha$  return val
    }
    Return val
}

```

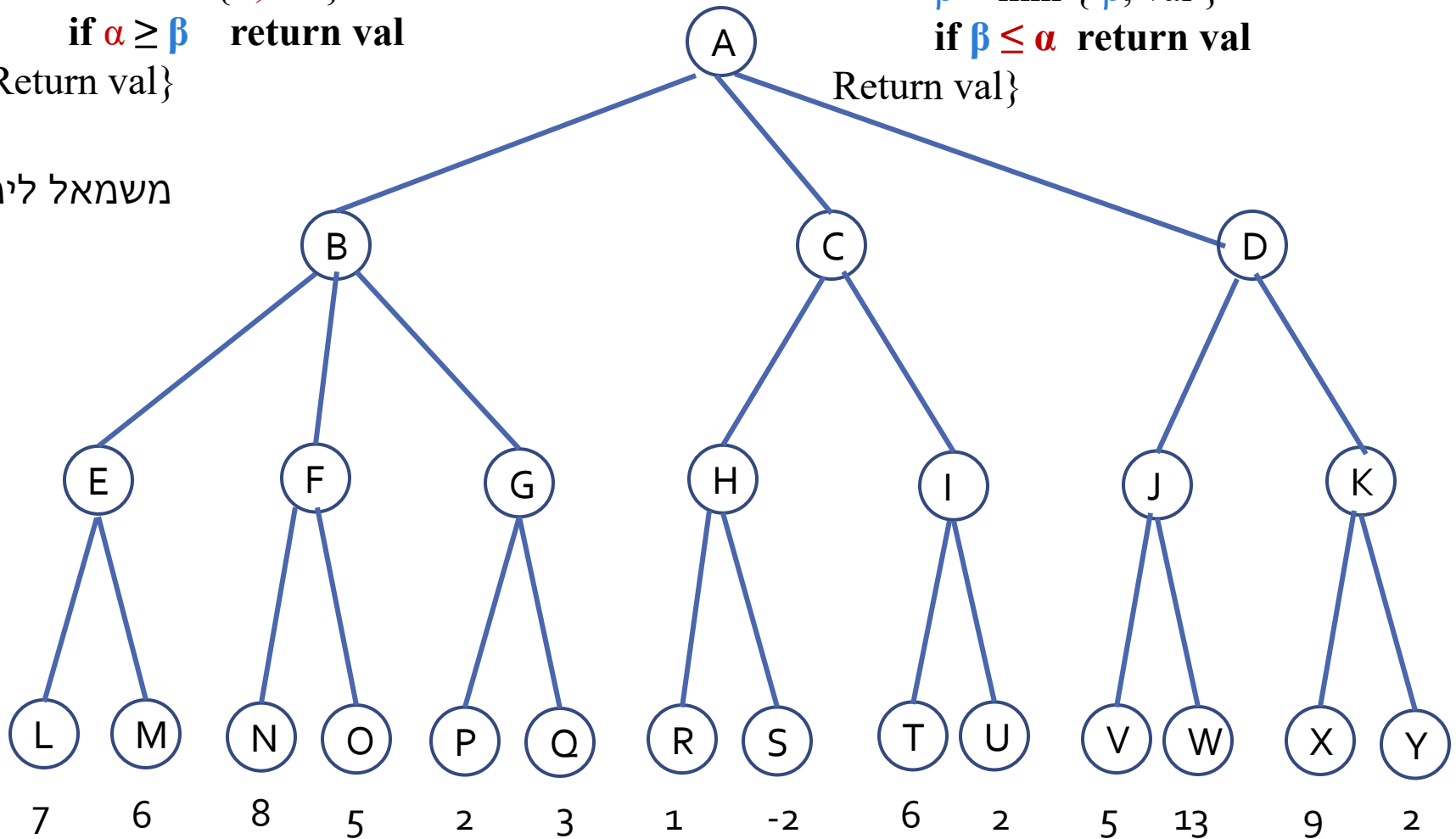
MaxValue(state, α , β)

```
{ val =  $-\infty$ 
  For all s in Successors(state)
    v = max {val, MinValue(s,  $\alpha$ ,  $\beta$ )}
     $\alpha$  = max { $\alpha$ , val}
    if  $\alpha \geq \beta$  return val
  Return val}
```

MinValue(state, α , β)

```
{ val =  $\infty$ 
  For all s in Successors(state)
    val = Min {v, MaxValue(s,  $\alpha$ ,  $\beta$ )}
     $\beta$  = min { $\beta$ , val}
    if  $\beta \leq \alpha$  return val
  Return val}
```

משמאל לימין



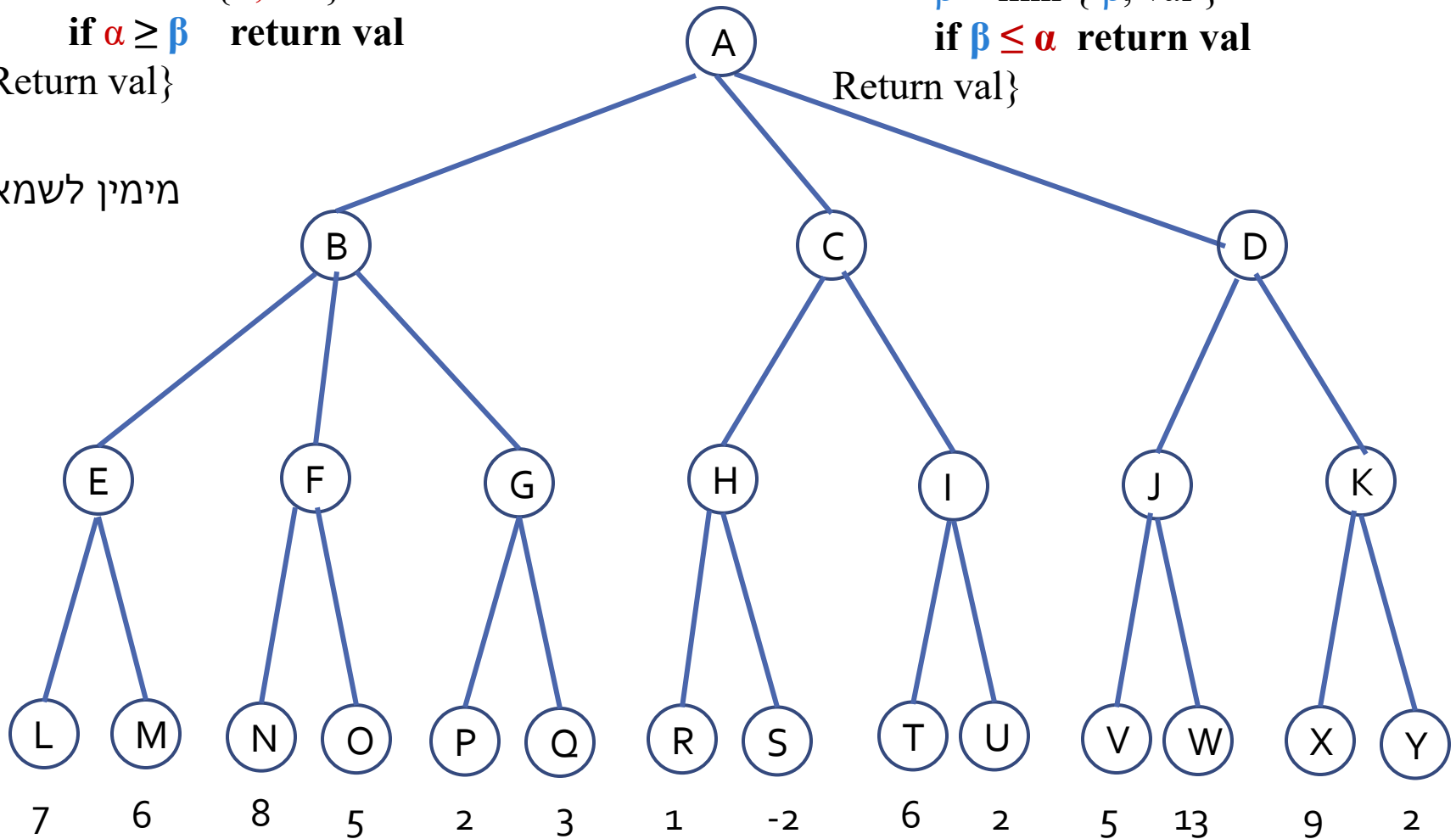
MaxValue(state, α , β)

```
{ val =  $-\infty$ 
  For all s in Successors(state)
    v = max {val, MinValue(s,  $\alpha$ ,  $\beta$ )}
     $\alpha$  = max { $\alpha$ , val}
    if  $\alpha \geq \beta$  return val
  Return val}
```

MinValue(state, α , β)

```
{ val =  $\infty$ 
  For all s in Successors(state)
    val = Min {v, MaxValue(s,  $\alpha$ ,  $\beta$ )}
     $\beta$  = min { $\beta$ , val}
    if  $\beta \leq \alpha$  return val
  Return val}
```

מימין לשמאל



$$\{ \text{val} = \infty$$
$$\text{val} = \text{Min} \{v, \text{MaxValue}(s, \alpha, \beta)\}$$

if $\beta \leq \alpha$ return val

Return val}

$$\{ \text{val} = -\infty$$
$$v = \max\{val, \text{MinValue}(s, \alpha, \beta)\}$$
$$\alpha = \max\{\alpha, \text{val}\}$$

if $\alpha \geq \beta$ return val

Return val}



תכונות גיזום α - β

★ הגיזום אינו משנה את התשובה הסופית !

★ יעילות הגיזום תלויה בסדר בדיקת הקודקודים:

★ לו היינו מצליחים לבדוק הקודקודים לפי סדר מושלם, סיבוכיות הזמן של Minmax עם השיפור של גיזום α - β היתה $O(b^{m/2})$ ← ניתן לחפש עד לעומק פי 2!!!

★ אם הצאצאים נבדקים לפי סדר רנדומאלי הזמן הדרוש הוא $O(b^{3m/4})$.

★ זוהי דוגמא ל metareasoning הסקת מסקנות לגבי הסקת מסקנות – חשיבה אילו חישובים כדאי לבצע.

דרך ב' לשיפור - Transposition Table

• ראינו בנושא חיפושים שמצבים חוזרים יוצרים זמן אקספוננציאלי. במשחקים לרוב יש מצבים חוזרים רבים ולכן כדאי לשמור את ערכי הmaxmin שלהם בטבלת גיבוב בפעם הראשונה שחישבנו אותם, למנוע חישוב כפול.

• זוהי טבלת ה transposition שמתפקדת כמו ה closed list.

• כשיש מיליוני קודקודים, זה לא ישים לשמור כל החוזרים בטבלה, ויש אסטרטגיות אלו קודקודים כדאי לשמור.

דרך ג' Imperfect Real Time Decision

למרות הגיזום אלג' החיפוש α - β עדיין צריך להגיע למצבי סיום בחלק מהעץ. העומק לרוב אינו מעשי כי צריך לקבל החלטה בזמן אמת. פתרון אפשרי נוסף:

- cutoff test: חותכים ברמות מסוימות ומתייחסים לקודקודים כעלים.

- evaluation function במקום לחשב ערך $\min\max$ אמיתי, מעריכים את התועלת המשוערת של הקודקוד.

Evaluation Function

פונקציית ההערכה מחזירה את התועלת הצפויה למשחק מהקודקוד הנתון. (דומה לפונק' היוריסטית).

1. החישוב צריך להיות מהיר.
 2. עבור מצבי סיום אמיתיים הפונק' צריכה להחזיר ערך התועלת האמיתי.
- רוב פונק' ההערכה מתייחסות לתכונות של מצבים. יש הנותנות ציון למצבים לפי ידע מוקדם של אחוזי הצלחה ממצבים בעלי תכונות אלו. דורש ניסיון רב.

Evaluation Function

פונק' הערכה רבות נותנות משקל לכל תכונה ויוצרות ציון משוקלל

- $Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$

- e.g., $w_1 = 9$ and $w_2 = 1$: למשל בשחמט:

$$f_1(s) = (\text{num of white queens}) - (\text{num of black queens}),$$

$$f_2(s) = (\text{number of pawns}) - (\text{number of black pawns}),$$

יש גם הגדרות שמציון מסוים סביר להניח שננצח, מציון אחר

בוודאי ננצח...

Cutting Off search

MinimaxCutoff is identical to *MinimaxValue*

except

if TerminalTest(state)
 return Utility(state)



if Cutoff(state)
 return Eval(state)

Cutting Off search

★ מתי נחתוך? או לפי עומק קבוע מראש

או לפי iterative deepening כלומר נחזיר צעד שהוחלט ע"י החיפוש
הכי עמוק שהספקנו.

★ ניתן לחתוך גם כשיש מספר אפשרויות שוות הערכה או כשיש אחת
שהיא בוודאות טובה יותר מהאחרות (single extension).

★ צריך להיזהר לחתוך במצבים יציבים, כשלא צפוי שינוי משמעותי
בשלים הבאים, אחרת עלולים לטעות בשל הקרוב של פונק' ההערכה.

משחקים בפועל

globes.co.il/news/article.aspx

הבינה המלאכותית עשתה דרך ארוכה בעשרים השנים שחלפו מאז הביס כחול עמוק, המחשב של יבמ, את גארי קספרוב ואף בשש השנים שעברו מאז ניצח ווטסון את קן ג'ינגס במשחק הטריוויה ג'פרדי. מחשבים הביסו שחקנים אנושיים מובילים בשחמט, שש בש ופוקר.

האנושות צריכה להודות לחוקרי מכון מסצ'וסטס לטכנולוגיה (MIT) על הניצחון במשחק Melee, אך זה אינו משחק הווידיאו היחיד שזוכה לזמן משחק רב מצדן של מכונות לומדות. תוכנות בינה מלאכותית פיצחו את Super Mario Bros, המשחקים הראשונים שפיתחה אטארי בשלבים מוקדמים, כגון Space Invaders, משחקי ארקייד מובילים כמו פקמן ו-Mortal Kombat, ואפילו את אנגרי בירדס. האופטימיים טוענים כי הבינה המלאכותית יכולה לסייע בפתרון הבעיות הקשות ביותר של העולם, כולל מחלת הסרטן והשינוי האקלימי. אז למה מקדישות מערכות הבינה המלאכותית זמן כה רב לגיימינג?

שם המשחק הוא נתונים. המשחקים מאפשרים לתוכנות בינה מלאכותית להתמודד עם בעיות לוגיות מורכבות מהסוג שקיים בעולם האמיתי - חוסר ודאות, ניהול מו"מ, הולכת שולל, שיתוף פעולה - בסביבה מבוקרת בקפדנות, לדברי ולד פיריוו, שהיה חבר בצוות שפיצח את Melee. חוקרים יכולים להשיק תוכנות בינה מלאכותית באמצעות בעיות משחקי ווידיאו פשוטות יחסית, לבצע אלפי או מיליוני ניסויים, ולאחר מכן לעבור בהדרגה לאתגרים מורכבים יותר.

"במשחקים אתה יכול ליצור כמה נתונים שאתה רוצה", אומר דמיס הסאביס, מנכ"ל DeepMind Technologies, חברת הבינה המלאכותית הפועלת מלונדון ונמצאת בבעלות החברה-האם של גוגל, אלפבית. "אתה צריך להגיע לנקודה האופטימלית - לא קשה מדי ולא קל מדי לאלגוריתמים הנוכחיים שלך".

סביבות משחק הן אידיאליות למה שידוע כלמידת חיזוק (reinforcement learning),

גלובס. למילים יש ערך

נרשמים לגלובס בדיגיטל ומתעדכנים בכל מה שקורה עכשיו בשוק

[יש לי מיני גלובס בדיגיטל](#)



להרשמה

משחקים דטרמיניסטים בפועל שחמט



1997 תוכנת Deep Blue של
IBM ניצחה את אלוף העולם
גרי קספרוב במשחק בן 6
מערכות.

(מחשב מקבילי 80 מעבדים)

Deep Blue ניצחה את גרי קספרוב

★ המחשב חיפש 126 מיליון קודקודים בשניה בממוצע.

★ פיתח עד 30 ביליון מצבים עבור צעד.

★ תמיד הגיע לעומק 14

★ השתמש ב *iterative deepening* של α - β עם טבלת גיבוב.

★ השתמש במבני נתונים של מידע עזר:

- ספר פתיחות עם 4000 אפשרויות
- 700,000 משחקים של אלופים להסקת מסקנות
- כל הסיומים האפשריים עם 5 כלים על הלוח

★ פונקציית הערכה של 8000 תכונות

★ סה"כ קיבל אפקט של חיפוש לעומק של 40 !



משחקים דטרמיניסטים בפועל דמקה

1994 התוכנית Chinook ניצחה את אלוף העולם במשך
40 שנה Marion Tinsley.

★ עבדה על PC רגיל .

★ השתמשה בחיפוש α - β .

★ השתמשה במבנה נתונים של סיום משחק מושלם החל
ממצבים עם 8 כלים או פחות שכלל 444 ביליון מצבים.

Deterministic games in practice

Go משחק פופולרי באסיה. גורם התפצלות של 361.

🌟 לאחרונה נוצח שחקן מנוסה.

הפניה•

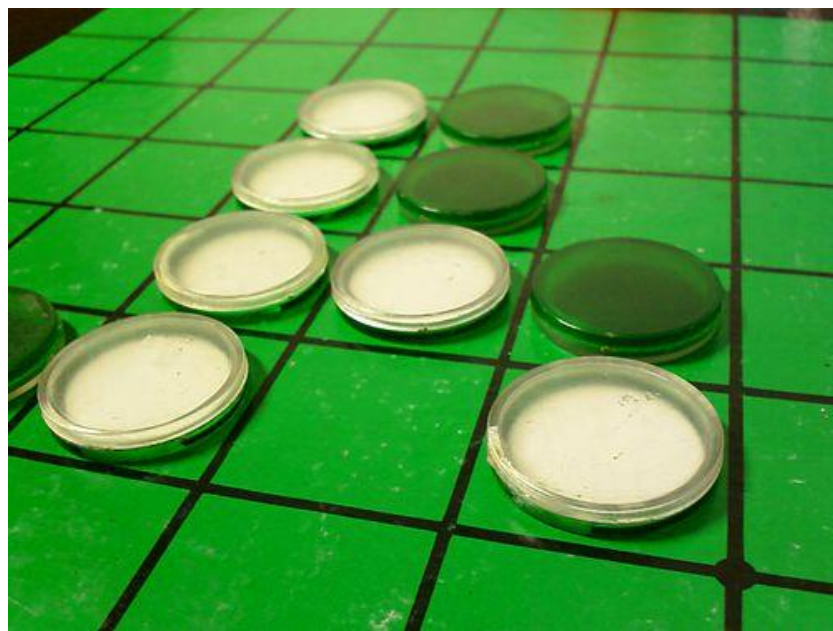


Deterministic games in practice

• רברסי (אותלו) יש לרוב 5-15 צעדים חוקיים

(בשח 35). היה צורך לצבור ניסיון.

Logistelo 1997 ניצחה את אלוף העולם.



שש בש



★ בשש בש ישנו רכיב של חוסר ודאות, כיוון שמטילים קוביות.

★ חיפוש עמוק אינו אפשרי ולכן משקיעים יותר בפונקציות ההערכה.

★ TD-Gammon 1992 עובדת עם רשתות נוירונים ופיתחה פונקציית הערכה מדויקת שמצריכה חיפוש של 2-3 רמות והיא מדורגת בין 3 השחקנים הטובים בעולם.

Volume 6, Issue 2

March 1994



[< Previous Article](#) [Next Article >](#)

Article Contents

[Abstract](#)

March 01 1994

TD-Gammon, a Self-Teaching Backgammon Program, Achieves Master-Level Play

Gerald Tesauro

[> Author and Article Information](#)

Neural Computation (1994) 6 (2): 215–219.

<https://doi.org/10.1162/neco.1994.6.2.215> [Article history](#) 

 PDF  Share  Tools 

Abstract

TD-Gammon is a neural network that is able to teach itself to play backgammon solely by playing against itself and learning from the results, based on the TD(λ) reinforcement learning algorithm (Sutton 1988). Despite starting from random initial weights (and hence random initial strategy), TD-Gammon achieves a surprisingly strong level of play. With zero knowledge built in at the start of learning (i.e., given only a “raw” description of the board state), the network learns to play at a strong intermediate level. Furthermore, when a set of hand-crafted features is added to the network’s input representation, the result is a truly staggering level of performance: the latest version of TD-Gammon is now estimated to play at a strong master level that is extremely close to the world’s best human players.

Issue Section: Notes

Issue Section: Notes

This content is only available as a PDF.

[קישור למאמר](#)

 PDF



פוקר

בינואר 2017 תוכנת בינה מלאכותית בשם "ליברטוס" Liberatus ריסקה באופן שיטתי את מיטב שחקני הפוקר העולמיים במשחק ה**טקסס הולדם** נו לימיט, בתחרות שנערכה בקזינו Riverss שבפיטסבורג.

★ ארבעה מקצועני פוקר עולמיים מהשורה הראשונה התחייבו לשחק 120 אלף ידי פוקר הדס-אפ (אחד על אחד) מול רובוט הפוקר שפיתחו צוות חוקרים מאוניברסיטת Carnegie Mellon .

פוקר - המשך

★ המקצוענים הסכימו להקדיש מזמנם ומרצם בתמורה לתהילה ולסיכוי לזכות בפרס של 200 אלף דולר אם יביסו את הבינה המלאכותית.

★ בינת הפוקר המלאכותית ניצחה את המקצוענים בפער בלתי-נתפס של 1.7 מיליון צ'יפים. מדובר ב-17 אלף בליינדים גדולים שהורווחו על-ידי הרובוט, כ-14 בליינדים גדולים על כל 100 ידי פוקר ששוחקו. פער עצום שכזה נחשב למובהק סטטיסטית ומעיד בבירור על יכולת הפוקר העצומה של הרובוט.

★ לאחר מרתון מתיש של 20 ימי משחק רצופים, רובוט הפוקר הסתגל לאסטרטגיית המשחק של השחקנים, ניצל את החולשות שלהם ומתקן את הטעויות אותן מצאו המקצוענים. מיום ליום רובוט הפוקר נהפך לחכם יותר ועשה פחות טעויות.

Summary

- ★ Games are fun to work on!
- ★ They illustrate several important points about AI
- ★ perfection is unattainable → must approximate
- ★ good idea to think about what to think about